

Customer Churn Analytics – Complete SQL Script

CREATE TABLE: Define the structure of the dataset

```
CREATE TABLE bank_churn (  
    row_number INT,  
    customer_id BIGINT,  
    surname TEXT,  
    credit_score INT,  
    geography TEXT,  
    gender TEXT,  
    age INT,  
    tenure INT,  
    balance NUMERIC,  
    num_of_products INT,  
    has_cr_card BOOLEAN,  
    is_active_member BOOLEAN,  
    estimated_salary NUMERIC,  
    exited BOOLEAN  
);
```

DATA PREVIEW & VALIDATION

```
SELECT * FROM bank_churn LIMIT 10;  
SELECT COUNT(*) FROM bank_churn;  
SELECT * FROM bank_churn  
WHERE row_number IS NULL OR customer_id IS NULL OR credit_score IS  
NULL;  
SELECT DISTINCT gender FROM bank_churn;  
SELECT DISTINCT geography FROM bank_churn;  
SELECT MIN(age), MAX(age), MIN(balance), MAX(balance) FROM bank_churn;
```

ALTER TABLE: Add/Modify/Drop Columns

```
ALTER TABLE bank_churn ADD COLUMN account_open_date DATE;  
ALTER TABLE bank_churn ALTER COLUMN surname TYPE VARCHAR(100);  
ALTER TABLE bank_churn DROP COLUMN account_open_date;  
ALTER TABLE bank_churn RENAME COLUMN estimated_salary TO salary;
```

BASIC QUERIES: Filtered Data Retrieval

```
SELECT * FROM bank_churn WHERE geography = 'France';
SELECT customer_id, age, balance FROM bank_churn WHERE age > 50 AND
balance > 100000;
```

INSERT, UPDATE, DELETE OPERATIONS

```
INSERT INTO bank_churn (
    row_number, customer_id, surname, credit_score, geography,
    gender, age, tenure, balance, num_of_products,
    has_cr_card, is_active_member, salary, exited
)
VALUES (
    10001, 15899999, 'Sharma', 720, 'Germany',
    'Male', 35, 5, 50000.00, 2,
    TRUE, TRUE, 75000.00, FALSE
);
```

```
UPDATE bank_churn SET exited = TRUE WHERE is_active_member = FALSE;
DELETE FROM bank_churn WHERE customer_id = 15899999;
```

FEATURE ENGINEERING: Create Age Group Buckets

```
ALTER TABLE bank_churn ADD COLUMN age_group TEXT;
UPDATE bank_churn
SET age_group = CASE
    WHEN age < 30 THEN 'Under 30'
    WHEN age BETWEEN 30 AND 45 THEN '30-45'
    WHEN age BETWEEN 46 AND 60 THEN '46-60'
    ELSE '60+'
END;
```

SEGMENTATION ANALYSIS BY DEMOGRAPHICS

```
SELECT age_group, COUNT(*) AS total_customers,
    SUM(CASE WHEN exited THEN 1 ELSE 0 END) AS exited_customers,
    ROUND(100.0 * SUM(CASE WHEN exited THEN 1 ELSE 0 END) /
COUNT(*), 2) AS churn_rate_percent
FROM bank_churn
GROUP BY age_group
ORDER BY churn_rate_percent DESC;
```

```
SELECT gender,
    COUNT(*) AS total_customers,
```

```

        SUM(CASE WHEN exited THEN 1 ELSE 0 END) AS exited_customers,
        ROUND(100.0 * SUM(CASE WHEN exited THEN 1 ELSE 0 END) /
COUNT(*), 2) AS churn_rate_percent
FROM bank_churn
GROUP BY gender
ORDER BY churn_rate_percent DESC;

SELECT geography,
        COUNT(*) AS total_customers,
        SUM(CASE WHEN exited THEN 1 ELSE 0 END) AS exited_customers,
        ROUND(100.0 * SUM(CASE WHEN exited THEN 1 ELSE 0 END) /
COUNT(*), 2) AS churn_rate_percent
FROM bank_churn
GROUP BY geography
ORDER BY churn_rate_percent DESC;

```

FINANCIAL BEHAVIOR ANALYSIS

```

SELECT exited,
        ROUND(AVG(balance), 2) AS avg_balance,
        ROUND(AVG(salary), 2) AS avg_salary
FROM bank_churn
GROUP BY exited;

SELECT has_cr_card,
        COUNT(*) AS total_customers,
        SUM(CASE WHEN exited THEN 1 ELSE 0 END) AS exited_customers
FROM bank_churn
GROUP BY has_cr_card;

```

SUMMARY STATISTICS: Mean, Median, StdDev

```

SELECT
        ROUND(AVG(credit_score), 2) AS avg_credit_score,
        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY credit_score) AS
median_credit_score,
        MIN(credit_score) AS min_credit_score,
        MAX(credit_score) AS max_credit_score,
        ROUND(STDDEV(credit_score), 2) AS stddev_credit_score,

        ROUND(AVG(balance), 2) AS avg_balance,
        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY balance) AS
median_balance,
        MIN(balance), MAX(balance),
        ROUND(STDDEV(balance), 2) AS stddev_balance
FROM bank_churn;

```

WINDOW FUNCTIONS: Top Customer Identification

-- a. Top 10 Non-Churned Customers by Value

```
SELECT *,  
       RANK() OVER (ORDER BY balance DESC, salary DESC) AS  
customer_rank  
FROM bank_churn  
WHERE exited = FALSE  
LIMIT 10;
```

-- b. Top 5 Churned Customers with High Balance (Retention Risk)

```
SELECT *,  
       RANK() OVER (ORDER BY balance DESC) AS risk_rank  
FROM bank_churn  
WHERE exited = TRUE  
LIMIT 5;
```

-- c. Top 5 Loyal High-Value Customers to Offer Reduced Interest Rate

```
SELECT *,  
       RANK() OVER (ORDER BY balance DESC, credit_score ASC) AS  
offer_priority  
FROM bank_churn  
WHERE exited = FALSE  
LIMIT 5;
```